

EVMAdapter 구현 분석 + XRPL Mock 교체

ethers.js · ABI · 이벤트 구독 · 멀티체인 전환
강의 55min + 실습

EVMAdapter

IBlockchainAdapter

XRPL Mock

COINCRAFT
ACADEMY

S14 → S15 연결

FROM INTERFACE TO IMPLEMENTATION

S14 — 인터페이스 설계

- `IBlockchainAdapter` 인터페이스 정의
- Strategy Pattern — 어댑터 교체 설계
- Factory Pattern — 생성 책임 분리
- `ChainAdapterRegistry` — 멀티체인 동시 운영

S15 — 실제 구현 분석

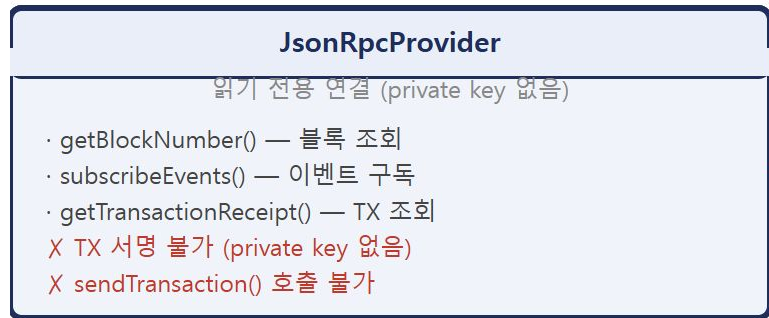
- `EVMAAdapter` 구현체 전체 분석
- ethers.js `Provider` vs `Wallet`
- ABI — 컨트랙트 호출의 핵심
- 이벤트 구독 + XRPL Mock 교체



오늘 목표 — 인터페이스 설계 결정이 왜 그렇게 됐는지 구현체를 통해 역으로 이해한다.

ethers.js Provider vs Wallet

TWO CORE OBJECTS — READ-ONLY vs READ-WRITE



핵심 — Wallet 은 Provider 위에 서명 능력을 덧씌운 것. 읽기는 Provider, 쓰기는 Wallet.

왜 Provider와 Wallet을 분리하는가

SECURITY PRINCIPLE — MINIMAL KEY EXPOSURE

EVMAAdapter 생성자

```
constructor(config: {  
  rpcUrl:      string;  
  chainId:     string;  
  privateKey?: string; // optional  
}) {  
  this.provider = new JsonRpcProvider(config.rpcUrl);  
  this.wallet = config.privateKey  
    ? new Wallet(config.privateKey, this.provider)  
    : null; // privateKey 없으면 read-only  
}
```

Phase 1 (현재) — 프로세스 내 분리

ChainEventListener → privateKey 없이 EVMAAdapter 생성

→ `wallet = null` → 이벤트 구독·블록 조회만 수행

IssuerService → mint는 VASP(월렛원)가 대신 서명·전송

→ EVMAAdapter에 privateKey 자체가 불필요

→ 같은 프로세스 안에서 동작 (서버 격리 없음)

Phase 3 (직접 Custody) — 서버 격리 필요

이벤트 구독 서버: privateKey 없이 EVMAAdapter → read-only

서명 서버: privateKey 주입 → `wallet = new Wallet(...)`

→ mintNFT · sendTransaction 활성화

→ 서명 서버는 HSM 등 별도 격리 환경에서만 운영

Phase 1 — 서버 격리 없음. privateKey optional 패턴으로 프로세스 내 역할 구분만 존재.

Phase 3 — 비로소 서버 격리가 의미를 가짐. 이벤트 서버가 해킹당해도 privateKey는 없다.

생성자 — 3가지 보안 원칙

CONSTRUCTOR SECURITY — THREE HARD RULES

원칙	이유
내부망 RPC 노드만 사용	public RPC(infura.io 등)는 TX 내용이 외부에 노출. 쿼리 패턴·지갑 주소 추적 가능
privateKey 환경변수 주입	코드 하드코딩 → git 이력에 키 유출. EVM_SIGNER_KEY 환경변수로만 주입
privateKey 없으면 read-only	이벤트 구독·잔액 조회 전용 인스턴스 분리 가능. 서명 키 노출 범위 최소화

read-only 모드에서 sendTransaction 호출 시

```
// EVMAAdapter.ts:137
async sendTransaction(params: ContractCallParams): Promise<TransactionReceipt> {
  if (!this.wallet) throw new Error('EVMAAdapter: read-only mode, no private key');
  // ...
}
```

조용한 실패 방지 — null 체크 후 즉시 명시적 에러. read-only 인스턴스가 실수로 TX 전송을 시도하면 즉각 발견.

Private RPC를 써야 하는 이유 (1/4)

① RATE LIMIT — PUBLIC RPC IS TOO SLOW FOR PRODUCTION

공개 RPC 한도 (Infura 무료)

100,000 req/day

→ 초당 약 1.1 req

- `subscribeEvents` → 블록마다 이벤트 수신
- `queryEvents` → Finalized 범위 배치 조회
- `getReceipt` → PENDING TX마다 주기적 폴링
- `getBlockNumber` → Lag 모니터링

공개 RPC 사용 시

트래픽 조금만 늘어도 한도 초과

→ 이벤트 누락

→ 원장 불일치

"RPC 한도 초과로 NFT 발행 미감지"

금융 시스템에서 허용 불가

Private RPC (월렛원 BaaS)

전용 인증 엔드포인트

Rate limit 계약 보장

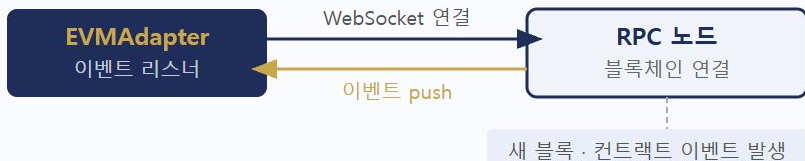
월렛원이 TX 서명 담당

우리는 read-only 쿼리 수행

Private RPC를 써야 하는 이유 (2/4)

② WEBSOCKET STABILITY — EVENT SUBSCRIPTION RELIABILITY

subscribeEvents 내부 동작



내부적으로 `eth_subscribe` (WebSocket) 사용
연결이 끊기면 이벤트 수신 중단

공개 RPC WebSocket 문제

- 무료 티어: WebSocket 미지원 또는 연결 수 제한
- 연결이 자주 끊김 → 재연결 로직 필요
- 재연결 시간 동안 이벤트 누락
- missed event 복구 부담 증가

missed event 복구 — 재시작 시 DB의 마지막 처리 블록부터
`queryEvents()` 로 보충. 안정적 WebSocket 연결이 전제.

Private RPC — WebSocket 연결 수 보장. 안정적 구독으로 재연결 빈도 최소화.

Private RPC를 써야 하는 이유 (3/4)

③ SLA — SERVICE LEVEL AGREEMENT FOR FINANCIAL SYSTEMS

공개 RPC — SLA 없음

- 가용성 보장 없음
- 장애 시 연락처 없음
- 다운타임 = 이벤트 수신 중단
- 내부 통제 위반 요소 가능

전용 RPC (월렛원 BaaS) — SLA 계약

- 99.9% 가용성 SLA 계약 가능
- 장애 시 에스컬레이션 경로 존재
- 점검 일정 사전 공지
- 금융 규제 환경 인프라 요건 충족

금융 규제 관점 — 인프라 SLA 없이 운영하는 것은 내부 통제 위반 요소가 될 수 있다. 외부 감사 시 SLA 계약서 제출 필요.

Phase 1 Private RPC 의미 — 노드를 직접 운영하는 것이 아니라, 월렛원이 BaaS로 제공하는 **전용 인증 엔드포인트**를 사용한다.

Private RPC를 써야 하는 이유 (4/4)

④ QUERY PRIVACY — FINANCIAL DATA EXPOSURE RISK

공개 RPC 운영자가 볼 수 있는 것

- 우리가 모니터링하는 컨트랙트 주소
- 조회하는 지갑 주소 패턴
- 어떤 이벤트를 얼마나 자주 확인하는지
- TX 조회 빈도 → 사용자 수 추정 가능

노출 시 위험

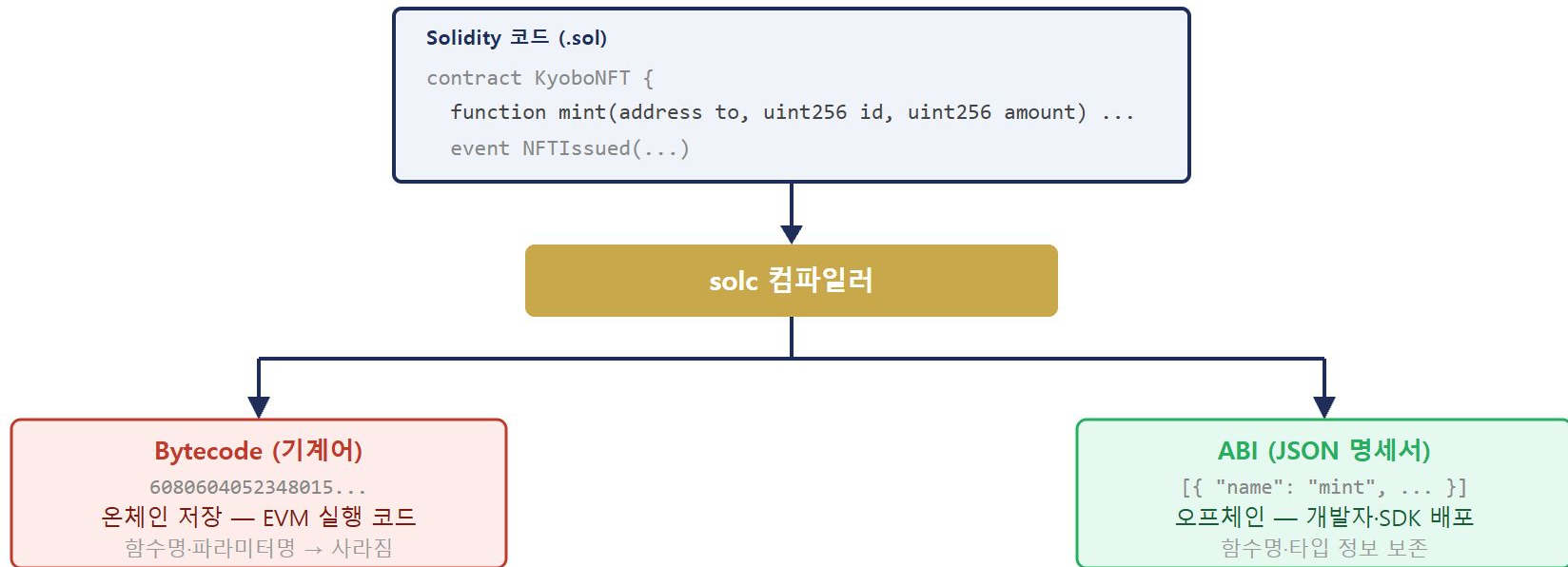
- 교보 고객 지갑 주소 패턴 → 제3자 추적 가능
- 금융 데이터 프라이버시 위반 가능성
- 경쟁사가 우리 컨트랙트 모니터링 패턴 파악
- 규제 기관 데이터 처리 요건 위반

read-only라도 private이어야 하는 이유 — TX 서명 없이 읽기만 해도 쿼리 자체가 민감한 금융 정보를 담고 있다. 공개 인프라를 통해 외부로 노출하면 안 된다.

결론 — Private RPC는 TX 서명을 위해서가 아니라 **운영 안정성 · SLA · 프라이버시**를 위해 필요하다.

ABI란 무엇인가

SOLIDITY COMPILATION — TWO OUTPUTS



ABI란 무엇인가 — Bytecode 경로

BYTECODE → DEPLOYMENT TX → ETHEREUM BLOCKCHAIN

배포 TX 구조

배포 트랜잭션:

to: (없음, 컨트랙트 생성)

data: 0x6080604052348015...

(bytecode 전체)

이더리움이 처리:

- 컨트랙트 주소 생성
- 0xKyoboNFT... = 주소 확정
- bytecode → 해당 주소에 영구 저장
- EVM이 실행 가능한 상태

왜 함수명이 사라지는가 — EVM은 함수 선택자(4바이트 해시)로만 구분. `mint` → `keccak256("mint(address,uint256,uint256)")[:4]` = 4바이트

배포 후 — bytecode는 블록체인에 영구 저장. 컨트랙트 주소만 알면 누구나 호출 가능. 하지만 어떻게 호출해야 하는지(파라미터 타입 등)는 ABI가 없으면 알 수 없다.

bytecode ≠ 소스코드 — 블록체인에는 컴파일된 기계어만 올라간다. 소스코드(.sol)는 검증 목적으로 별도 등록(Etherscan 등). 필수 아님.

ABI란 무엇인가 — ABI 경로

ABI → SDK → CALLDATA ENCODING → CONTRACT CALL

ethers.js + ABI + 컨트랙트 주소

ABI로 calldata 인코딩

0xa9059cbb 000..Alice 000..1001 ...

TX 전송

→ 컨트랙트가 calldata 해석 → 실행

calldata 구조

함수 선택자 4바이트 — keccak256(함수시그니처)[0:4]

인자 1 address — 32바이트 패딩

인자 2 uint256 — 32바이트 패딩

인자 3 uint256 — 32바이트 패딩

ABI 없으면 불가능한 것

- 함수 시그니처 모름 → 선택자 계산 불가
- 인자 타입 모름 → 바이트 인코딩 불가
- calldata 생성 불가 → 컨트랙트 호출 불가
- 이벤트 파싱 불가 → 로그 해석 불가

ABI = 컨트랙트 명세서 — 블록체인에는 기계어만 있다. ABI가 있어야 "어떤 함수를, 어떤 타입의 인자로 호출하는지" 알 수 있다.

ABI — 실제 JSON 형식

SOLC OUTPUT — JSON ABI OBJECT ARRAY

solc 컴파일러 출력 (JSON ABI)

```
[
  {
    "name": "mint",
    "type": "function",
    "stateMutability": "nonpayable",
    "inputs": [
      { "name": "to", "type": "address" },
      { "name": "id", "type": "uint256" },
      { "name": "amount", "type": "uint256" }
    ],
    "outputs": []
  },
  // 이벤트 정의 (indexed 필드 포함) - 다음 항목 참조
  { "name": "NFTIssued", "type": "event", "inputs": [ ... ] }
]
```

함수 하나 = 오브젝트 하나 — 필드가 많고 상황. 컨트랙트 함수가 10개면 10개의 오브젝트.

indexed 필드 — 이벤트 파라미터가 `indexed: true` 이면 topic으로 저장 → 필터 쿼리 가능. `false` 면 data 영역에 저장.

다음 장 — ethers.js는 이 JSON을 더 짧은 문자열 형식으로 표현 가능. Human-readable ABI.

ABI란 무엇인가 — calldata

CALLDATA — THE ENCODED FUNCTION CALL IN A TX

일반 ETH 전송 vs 컨트랙트 호출

일반 ETH 전송:

to: 0xAlice
value: 1 ETH
data: (없음)

컨트랙트 함수 호출:

to: 0xKyoboNFT (컨트랙트 주소)
value: 0
data: 0xa9059cbb ← 함수 선택자 4바이트
000...0xAlice ← address to (32바이트)
000...000003E9 ← tokenId = 1001
000...0000001 ← amount = 1

data 필드 = calldata — TX의 `data` 필드에 들어가는 바이트열 전체. ethers.js가 ABI를 기반으로 자동 생성한다.

함수 선택자 계산

```
keccak256("mint(address,uint256,uint256)")[:4]
```

함수 시그니처의 해시 앞 4바이트. ABI 없으면 이 시그니처를 알 수 없다.

ethers.js가 자동으로 처리

```
contract.mint("0xAlice", 1001n, 1n)
```

↓ ABI 기반 calldata 생성 → TX data 필드 삽입

ABI란 무엇인가 — 핵심 정리

KEY POINTS — BYTECODE vs ABI + ANALOGY

	Bytecode	ABI
저장 위치	블록체인 온체인	오프체인 (개발자 보관)
역할	EVM이 실행하는 실제 코드	함수명·파라미터 타입 명세
사람이 읽을 수 있나	❌ 기계어	✅ JSON
없으면	컨트랙트 실행 불가	외부에서 호출 방법을 모름

비유 — 레스토랑 메뉴판

메뉴판(ABI): "파스타 주문 시 → 이름, 수량, 소스 선택 필요"

주문(TX 호출): `mint(address to, uint256 id, uint256 amount)`

주방(컨트랙트): 파라미터를 받아서 처리

메뉴판 없이 주문하면? → 주방이 파라미터를 어떻게 해석해야 할지 모름

Human-readable ABI (ethers.js 형식)

HUMAN-READABLE ABI — ETHERS.JS EXCLUSIVE STRING FORMAT

ERC-1155 최소 ABI — EVMAdapter.ts:19

```
// JSON ABI 대신 문자열 배열로 표현
const ERC1155_ABI = [
  'function mint(address to, uint256 id, uint256 amount)',
  'function mintBatch(address[] to, uint256[] ids, uint256[] amounts)',
  'function burn(address from, uint256 id, uint256 amount)',
  'function balanceOf(address account, uint256 id) view returns (uint256)',
];
```

ethers.js가 이 문자열을 파싱 → 내부적으로 JSON ABI와 동일한 구조로 변환

ethers.js 전용 — web3.js 등 다른 라이브러리는 JSON ABI만 지원. ethers.js를 쓸 때만 Human-readable 형식 사용 가능.

왜 최소 ABI인가

- 어댑터가 호출하는 함수만 선언
- 컨트랙트 전체 ABI = 불필요한 의존성
- 업그레이드 시 영향 범위 최소화

결과는 동일 — JSON ABI든 Human-readable ABI든 ethers.js 내부에서 동일하게 처리. 가독성만 다르다.

EVMAdapter 메서드 — Phase별 사용 구분

METHOD USAGE BY PHASE — READ-ONLY vs WRITE

메서드	Phase 1	Phase 3+	역할
<code>isConnected()</code>	✓ 사용	✓ 사용	RPC 연결 헬스체크
<code>getBlockNumber()</code>	✓ 사용	✓ 사용	현재 블록 번호 조회 (Lag 모니터링)
<code>getReceipt(txHash)</code>	✓ 사용	✓ 사용	TX 상태 폴링 (PENDING → MINED)
<code>subscribeEvents()</code>	✓ 사용	✓ 사용	온체인 이벤트 실시간 구독
<code>queryEvents()</code>	✓ 사용	✓ 사용	블록 범위 이벤트 배치 조회 (복구)
<code>mintNFT()</code>	✗ 미사용	✓ 사용	NFT 발행 (privateKey 필요)
<code>burnNFT()</code>	✗ 미사용	✓ 사용	NFT 소각 (privateKey 필요)
<code>sendTransaction()</code>	✗ 미사용	✓ 사용	컨트랙트 함수 직접 호출

Phase 1 — TX 서명·발행은 월렛원(VASP)이 담당. EVMAdapter는 이벤트 감지 · 상태 확인 역할만 한다.

Phase 3+ 메서드 — ABI 사용 시작

WRITE METHODS — ABI FIRST APPEARS HERE

Phase 1 메서드(`isConnected` , `getBlockNumber` , `getReceipt`)는 ABI를 사용하지 않는다 — Provider 직접 호출. ABI는 컨트랙트 함수를 호출할 때 부터 필요.

mintNFT → sendTransaction 위임

```
// EVMAdapter.ts:82
async mintNFT(params: MintParams): Promise<TransactionReceipt> {
  return this.sendTransaction({
    contractAddr: params.contractAddr,
    abi: ERC1155_ABI, // ← ABI 여기서 처음 사용
    method: 'mint',
    args: [params.to, params.tokenId, params.amount],
  });
}
```

고수준 vs 저수준 — `mintNFT` / `burnNFT` 는 파라미터를 ABI 호출 형식으로 변환만 한다. 실제 TX 전송 로직은 `sendTransaction()` 한 곳에만 있다.

requestId가 args에 없는 이유 — 컨트랙트 시그니처가 `mint(address, uint256, uint256)` 으로 고정. calldata에 추가 불가. 실제 중복 차단은 DB 레벨 requestId unique constraint로 수행.

ABI 사용 메서드 구분

ABI 불필요 → `isConnected` , `getBlockNumber` , `getReceipt`

ABI 필요 → `mintNFT` , `burnNFT` , `getBalance` , `subscribeEvents` , `queryEvents`

sendTransaction — TX 전송의 핵심

PHASE SEPARATION — VASP vs DIRECT CUSTODY

sendTransaction 코드 (EVMAAdapter.ts:136)

```
async sendTransaction(  
  params: ContractCallParams  
) : Promise<TransactionReceipt> {  
  // Phase 1: 이 줄에서 즉시 throw  
  if (!this.wallet)  
    throw new Error('read-only mode, no private key');  
  
  // Phase 3+: 아래 코드 실행  
  const iface = new Interface(params.abi);  
  const contract = new Contract(  
    params.contractAddr, iface, this.wallet  
  );  
  const tx = await contract[params.method](...params.args);  
  const receipt = await tx.wait(); // ← 문제  
  return { txHash: receipt.hash, ... };  
}
```

Phase 1 — VASP가 TX 처리

월렛원 API → submitMint() → txHash 반환

TxStateMachineService → VaspTxClient.getStatus() 폴링

EVMAAdapter.sendTransaction() 호출 없음

Phase 3 — 직접 Custody

privateKey 주입 → wallet 활성화

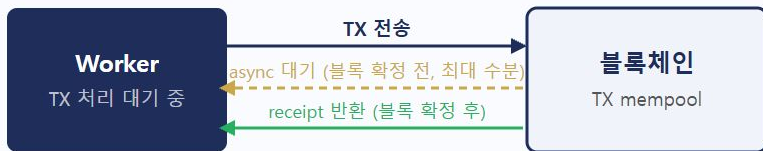
mintNFT() → sendTransaction() → 체인 직접 전송

현재 tx.wait() 구조 → 다음 장에서 문제 설명

sendTransaction — tx.wait() 문제

ASYNC SERIALIZATION — THROUGHPUT BOTTLENECK

현재 구조 (tx.wait() 사용)



Worker 다음 건 처리: 대기 중

- 100건 배치: 1건 확정 전 99건 시작 불가
- 처리량 = 1건 / 확정시간

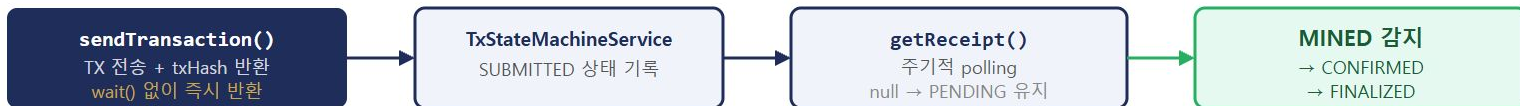
tx.wait()는 이벤트 루프를 차단하지 않는다 — async 함수이므로 다른 Promise는 계속 처리됨. 문제는 이 **async 흐름**이 TX 확정 전까지 다음 줄로 진행되지 못한다는 것.

타임아웃 없는 경우 — 네트워크 혼잡으로 TX가 mempool에 오래 머물면 이 async 함수가 영원히 재개되지 않을 수 있음.

sendTransaction의 반환 타입이 `Promise<TransactionReceipt>` — status, gasUsed 등 확정 후 데이터를 포함해야 함. tx.wait() 없이는 이 타입을 맞출 수 없음.

sendTransaction — 해결 방향

SOLUTION — TXHASH RETURN + POLLING SEPARATION



인터페이스 변경이 필요한 이유

- 현재: `sendTransaction()` → `Promise<TransactionReceipt>`
- 변경: `sendTransaction()` → `Promise<{ txHash: string }>`
- `IBlockchainAdapter` 인터페이스 자체를 바꿔야 함
- 인터페이스 재설계 없이는 `tx.wait()` 제거 불가

tx.wait()는 Phase 1 한계 — 블록 확정까지 스레드를 점유하며 대기. Phase 3에서는 `sendTransaction()`이 txHash만 반환하고, 확정 감지는 `TxStateMachineService`가 비동기로 담당하는 구조로 전환된다.

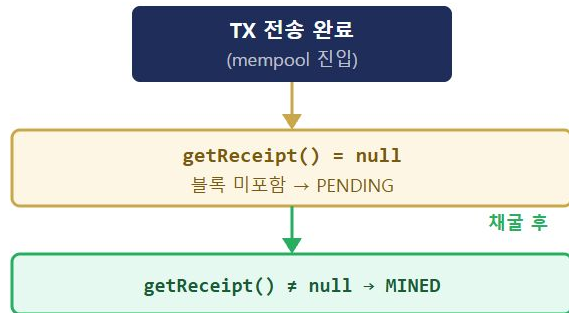
getReceipt()가 별도로 있는 이유 — `TxStateMachineService`가 `getReceipt(txHash)`를 주기적으로 호출해서 확정을 감지하는 구조로 전환하기 위한 사전 설계.

getReceipt — PENDING 상태 추적

NULL RETURN = PENDING — TXSTATEMACHINESERVICE INTEGRATION

getReceipt 코드 (EVMAdapter.ts:156)

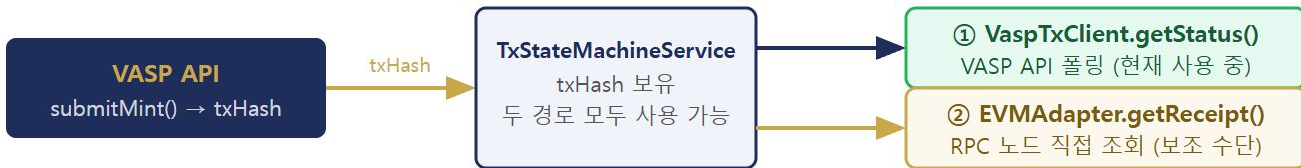
```
async getReceipt(  
  txHash: string  
) : Promise<TransactionReceipt | null> {  
  const receipt = await  
    this.provider.getTransactionReceipt(txHash);  
  
  if (!receipt) return null; // PENDING  
  
  return {  
    txHash: receipt.hash,  
    blockNumber: receipt.blockNumber,  
    blockHash: receipt.blockHash,  
    status: receipt.status === 1 ? 'success' : 'failed',  
    gasUsed: receipt.gasUsed,  
    timestamp: Date.now(),  
  };  
}
```



null의 의미 — RPC 노드에 TX는 알려져 있지만 블록에 아직 포함되지 않음. TxStateMachineService 가 이 null을 PENDING으로 해석하고 TIMEOUT 타이머를 관리.

Phase 1에서도 getReceipt()는 쓸 수 있다

DUAL STATUS CHECK PATH — VASP API + DIRECT RPC



Phase 1에서 ②도 즉시 사용 가능 — VASP가 txHash를 반환한 순간 부터 `getReceipt(txHash)` 도 동작. 두 경로 모두 txHash만 있으면 됨.

②의 장점 — 장애 격리

VASP API 장애 → `VaspTxClient.getStatus()` 응답 없음

→ `EVMAdapter.getReceipt()` 로 체인 직접 조회

→ TX 상태 확인 가능 → 장애 전파 차단

Phase 2 ChainEventListener 병행도 같은 이유

sendTransaction vs getReceipt 비교

SEND vs POLL — TWO COMPLEMENTARY METHODS

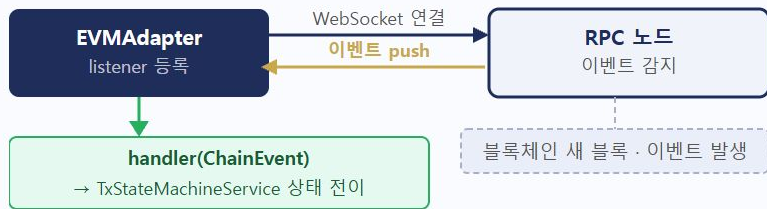
	sendTransaction	getReceipt
언제	TX를 처음 전송할 때	이미 전송된 TX 상태를 확인할 때
privateKey	필요 (wallet 없으면 throw)	불필요 (provider만 사용)
대기 방식	tx.wait() — async 흐름 직렬 대기	즉시 반환 (null or receipt)
반환값	TransactionReceipt (확정 후 데이터)	TransactionReceipt null
Phase 1 사용	✗ 미사용 (VASP가 처리)	✓ 보조 수단 (VaspTxClient 보완)
S13 연계	SUBMITTED 상태 진입	PENDING → MINED → CONFIRMED 전이 판단

설계 의도 — `sendTransaction` 은 TX를 "쓰는" 역할, `getReceipt` 는 "확인하는" 역할. 분리하면 TX 전송과 확정 추적을 독립적으로 구성할 수 있다.

이벤트 구독 패턴

HOW ON-CHAIN EVENTS ARE RECEIVED IN REAL TIME

WebSocket 구독 흐름



블록체인은 **push**하지 않는다 — 우리가 WebSocket으로 연결해서 구독(pull). RPC 노드가 이벤트 발생 시 연결된 클라이언트에게 알림.

unsubscribe 안 하면

- 서비스 종료 시 WebSocket 연결 살아있음
- 이미 종료된 handler가 이벤트를 받으려 함
- 메모리 누수 / 에러 로그 폭발
- GracefulShutdown 시간 초과

subscribeEvents (1/2)

IMPLEMENTATION — LISTENER ARRAY + CLOSURE PATTERN

subscribeEvents 코드 (EVMAdapter.ts:171)

```
async subscribeEvents(
  contractAddr: string, abi: unknown[],
  eventNames: string[],
  _fromBlock: number, // WebSocket은 fromBlock 지정 불가
  handler: (e: ChainEvent) => Promise<void>,
): Promise<() => void> {
  const contract = new Contract(contractAddr, new Interface(abi), this.provider);
  const listeners: Array<() => void> = [];
  for (const eventName of eventNames) {
    const listener = async (...args) => {
      const log = args[args.length - 1];
      await handler(
        this._toChainEvent(eventName, contractAddr, log, args)
      );
    };
    contract.on(eventName, listener);
    listeners.push(() => contract.off(eventName, listener));
  }
  return () => listeners.forEach(off => off());
}
```

_fromBlock이 무시되는 이유 — ethers.js WebSocket 방식은 연결 시점부터 수신. missed event 복구는 queryEvents() 가 담당.

클로저 패턴 — `contract.off(eventName, listener)` 를 클로저로 캡처해서 `listeners` 배열에 저장. 반환 함수 호출 한 번으로 모든 구독 해제.

subscribeEvents (2/2) — 여러 이벤트 동시 구독

MULTI-EVENT SUBSCRIPTION — ONE UNSUBSCRIBE FOR ALL

여러 이벤트를 한 번에 구독하는 패턴

```
eventNames = ['NFTIssued', 'NFTBurned', ...]
```

for loop

```
각 이벤트마다 contract.on(eventName, listener)  
→ () => contract.off() 클로저를 listeners[] 에 push
```

```
listeners = [off1, off2, off3]  
각 이벤트의 off() 클로저 배열
```

```
unsubscribe() → listeners.forEach(off => off()) → 모든 구독 해제
```

단일 **unsubscribe** 함수 — 이벤트 종류에 상관없이 반환된 함수 하나만 호출하면 전부 해제. GracefulShutdown 로직이 단순해짐.

사용 예시

```
const unsubscribe = await adapter.subscribeEvents(  
  contractAddr, abi,  
  ['NFTIssued', 'NFTBurned'],  
  latestBlock, handler,  
);  
  
process.on('SIGTERM', async () => {  
  unsubscribe(); // ← 모든 구독 일괄 해제  
  await server.close();  
});
```


subscribeEvents — GracefulShutdown 연결

GRACEFUL SHUTDOWN — SAFE PROCESS TERMINATION

강제 종료 (kill -9) vs Graceful Shutdown

강제 종료 (kill -9)

WebSocket 구독 해제 없이
프로세스 사망
→ 메모리 누수
→ 에러 로그 / TIME_WAIT 누적

vs

Graceful Shutdown

- ① 새 요청 수신 중단
- ② unsubscribe() → WebSocket 해제
- ③ 진행 중 이벤트 처리 완료 대기
- ④ DB 커넥션 반환, 소켓 닫기
- ⑤ 프로세스 종료

→ 상태 일관성 보장

금융 시스템에서 중요한 이유

TX 전송 중 강제 종료

- DB 상태: SUBMITTED (기록됨)
- 실제 TX: 브로드캐스트 완료 여부 불확실
- 재시작 시 중복 전송 위험

Graceful Shutdown

TX 처리 완료 또는 명시적 실패 기록 후 종료
→ 상태 일관성 보장

kubectl rolling update , docker stop , PM2 restart 모두 내부적으로 SIGTERM 먼저 → 일정 시간 후 SIGKILL. Graceful Shutdown 핸들러가 그 시간 안에 정리를 마쳐야 한다.

queryEvents — missed event 복구

BATCH EVENT QUERY — FINALIZED BLOCK RANGE ONLY

queryEvents 코드 (EVMAdapter.ts:197)

```
async queryEvents(  
  contractAddr: string,  
  abi: unknown[],  
  eventName: string,  
  fromBlock: number,  
  toBlock: number,  
) : Promise<ChainEvent[]> {  
  const contract = new Contract(contractAddr, iface, this.provider);  
  const logs = await contract.queryFilter(  
    contract.getEvent(eventName), fromBlock, toBlock  
  ); // 내부: eth_getLogs RPC 호출  
  return logs.map(log => this._toChainEvent(...));  
}
```

왜 Finalized 범위만?

MINED 기준 queryEvents → REORG 발생 시 이미 처리한 이벤트가 사라짐 → 원장 불일치

Finalized 기준

절대 불변 → 안전하게 배치 처리 가능

```
queryEvents(lastProcessedBlock, finalizedBlock)
```

subscribeEvents와의 역할 분리

subscribeEvents → 실시간 수신 (연결 시점~)

queryEvents → 재시작 후 누락 구간 보충

_toChainEvent (1/2) — 변환 코드

EVM RAW EVENT → CHAIN-AGNOSTIC CHAINEVENT

_toChainEvent 코드 (EVMAdapter.ts:216)

```
private _toChainEvent(  
  eventName: string,  
  contractAddr: string,  
  log: EventLog,  
  args: unknown[],  
): ChainEvent {  
  const named: Record<string, unknown> = {};  
  if (log.fragment) {  
    log.fragment.inputs.forEach((input, i) => {  
      named[input.name] = log.args?.[i];  
    });  
  }  
  return {  
    eventName, contractAddr,  
    txHash: log.transactionHash,  
    blockNumber: log.blockNumber,  
    logIndex: log.index,  
    args: Object.keys(named).length ? named : { raw: args },  
    raw: log, // 원본 EventLog 보존  
  };  
}
```

이 메서드의 역할 — EVM 전용 `EventLog` 를 체인 독립적인 `ChainEvent` 로 변환. 상위 레이어가 EVM 구조를 직접 알 필요 없게 한다.

raw 필드 사용 원칙

✅ `_toChainEvent()` 내부 에서 raw → args 변환

❌ `IssuerService` 등 외부 에서 `ChainEvent.raw` 직접 파싱

XRPL 교체 시 raw 구조가 완전히 달라짐

_toChainEvent (2/2) — fragment.inputs 역할

NAMED ARGS RECONSTRUCTION FROM ABI FRAGMENT

fragment.inputs가 하는 일

```
EVM Event:  
NFTIssued(address to, uint256 tokenId, uint256 amount)
```

fragment.inputs

```
inputs[0].name = 'to' inputs[1].name = 'tokenId'  
inputs[2].name = 'amount'
```

forEach 매핑

```
named = { to: '0xAlice...', tokenId: 1001n, amount: 1n }
```

ChainEvent.args = named (인덱스 대신 named 접근 가능)

왜 **named args**가 필요한가 — 원시 로그의 `args[0]`, `args[1]` 대신 `args.to`, `args.tokenId` 로 접근 가능. 인덱스 순서 오류 방지.

fragment 없는 경우 — `log.fragment` 가 없으면 `{ raw: args }` 풀백. ABI에 없는 이벤트 로그도 최소한 원본은 보존.

어댑터 교체 시 — XRPLAdapter의 `_toChainEvent` 는 XRPL 원본 TX를 ChainEvent로 변환. **named args** 구조는 동일. IssuerService 코드 변경 없음.

XRPLAdapter Stub — 구조

STUB IMPLEMENTATION — COMPILE-TIME CONTRACT ENFORCEMENT

XRPLAdapter stub

```
export class XRPLAdapter implements IBlockchainAdapter {
  readonly chainId = 'xrpl-mainnet';
  readonly chainType = 'XRPL' as const;

  constructor(_config: { wsUrl: string; seed?: string }) {}

  async isConnected() { return false; }
  async getBlockNumber() { return 0; }

  async mintNFT(_p: MintParams): Promise<TransactionReceipt> {
    throw new Error('XRPLAdapter: not implemented');
  }
  // ... 나머지 메서드 동일
}
```

implements IBlockchainAdapter — stub이지만 인터페이스 충족 여부가 컴파일 타임에 검증. 메서드 하나라도 빠뜨리면 TypeScript 컴파일 에러.

gasUsed가 없는 이유

`TransactionReceipt.gasUsed?: bigint` — optional

XRPL은 가스 개념 없음 → undefined 반환

상위 레이어에서 gasUsed에 의존하면 오류

stub의 가치 — 실제 XRPL RPC 없이도 인터페이스 교체 시나리오 검증 가능. Phase 3 XRPL 구현 전에 연동 구조를 미리 확인.

XRPLAdapter — 교체 시뮬레이션

ISSUERSERVICE CODE ZERO CHANGE — ADAPTER SWAP ONLY

issuer-service/src/index.ts — 교체 지점 (1곳)

```
// ㉠ 현재 (EVM)
const chainAdapter = new EVMAAdapter({
  rpcUrl: process.env.RPC_URL!,
  chainId: process.env.CHAIN_ID!,
  privateKey: process.env.OPERATOR_PRIVATE_KEY!,
});

// ㉡ XRPL로 교체 시 - 이 한 블록만 바꾼다
const chainAdapter = new XRPLAdapter({
  wsUrl: process.env.XRPL_WS_URL!,
  seed: process.env.XRPL_SEED!,
});

// ㉢ 이 아래는 한 줄도 바꾸지 않는다
const factory = new TokenIssuerFactory({ chainAdapter, ... });
const eventListener = new ChainEventListener(chainAdapter, ...);
```

실제 주입 경로 3곳 — 모두 같은 chainAdapter 참조

TokenIssuerFactory({ chainAdapter }) → issuerService 내부로 전달

ChainEventListener(chainAdapter, ...) → 이벤트 구독·missed 복구

chainAdapter.call(...) → 잔액 조회 등 read 호출

교체 시 변경 범위

index.ts 상단 new EVMAAdapter() → new XRPLAdapter() 1줄

.env에 XRPL_WS_URL / XRPL_SEED 추가

나머지 비즈니스 로직 — 변경 없음

S15 핵심 정리

KEY TAKEAWAYS

개념	핵심
Provider vs Wallet	읽기=Provider, 쓰기=Wallet. privateKey 없으면 read-only 모드 — 키 노출 범위 최소화
Private RPC 이유	서명과 무관. Rate Limit · WebSocket 안정성 · SLA · 쿼리 프라이버시
ABI 역할	컨트랙트 함수명·파라미터 명세. 없으면 calldata 생성 불가 → 호출 불가
tx.wait() 문제	async 흐름 직렬화 → 처리량 병목. TxStateMachineService + getReceipt() 폴링으로 분리 필요
getReceipt Phase 1	VaspTxClient.getStatus() 보완 수단. 월렛원 API 장애 시 체인 직접 조회 경로
subscribeEvents	listeners 배열 + 클로저 → 단일 unsubscribe 함수. GracefulShutdown 연결
XRPLAdapter stub	implements 강제 → 컴파일 타임 메서드 누락 감지. gasUsed optional로 체인 차이 흡수

한 줄 요약 — EVMAdapter는 인터페이스 설계 결정의 이유를 역으로 보여준다. Provider/Wallet 분리·ABI·구독 패턴 모두 S14의

`IBlockchainAdapter` 설계를 뒷받침하는 구현이다.

S16 예고 — TX 전송 시 발생하는 에러 처리 · Idempotency Guard 설계. requestId 기반 중복 TX 전송 차단.

실습 — EVMAdapter 구현 분석

EXERCISE OVERVIEW · S15_evm_lab.ts

실행 방법 (프로젝트 루트에서)

명령어	내용
<code>npm run exercise:s15</code>	전체 실행
<code>npm run exercise:s15:1</code>	[1] isConnected()
<code>npm run exercise:s15:2</code>	[2] getBlockNumber()
<code>npm run exercise:s15:3</code>	[3] XRPLMockAdapter 교체
<code>npm run exercise:s15:4</code>	[4] read-only 모드
<code>npm run exercise:s15:5</code>	[5] mintNFT + getBalance
<code>npm run exercise:s15:7</code>	[7] queryEvents

전제 조건

인터넷 연결 필요 — Sepolia 공개 RPC 사용
[1][2][3][4] 는 환경변수 없이 즉시 실행 가능

[5][7] 은 환경변수 필요
MOCK_CONTRACT_ADDR — MockERC1155 컨트랙트 주소
EVM_SIGNER_KEY — Sepolia 서명 키

파일 경로

course/exercises/M3/S15_evm_lab.ts

[1] isConnected() · [2] getBlockNumber()

EVMAdapter — Sepolia 연결 확인 · read-only 모드 동작 이해

[1] isConnected() — 무엇을 확인하나

항목	내용
실행 명령	<code>npm run exercise:s15:1</code>
내부 동작	provider로 <code>eth_blockNumber</code> RPC 호출 → 응답 오면 <code>true</code>
privateKey	불필요 — Provider만으로 충분
실패 시	RPC URL 오류 / 네트워크 단절 → <code>false</code> 반환

출력: `isConnected: true`

→ Sepolia 노드와 연결 확립됨

[2] getBlockNumber() — 무엇을 확인하나

항목	내용
실행 명령	<code>npm run exercise:s15:2</code>
내부 동작	provider → <code>eth_blockNumber</code> → 현재 채굴된 블록 높이
의미	블록체인이 살아있고 진행 중임을 숫자로 확인
활용	<code>queryEvents</code> 범위 계산 시 <code>currentBlock</code> 으로 사용

출력: `blockNumber: 84XXXXX`

→ 매 실행마다 다름 (실시간 블록)

공통 포인트 — 두 메서드 모두 `this.wallet` 없이 동작한다. EVMAdapter 생성 시 `privateKey`를 주지 않아도 읽기 전용 기능은 100% 사용 가능.

[3] XRPLMockAdapter — 인터페이스 직접 구현

IBlockchainAdapter를 처음부터 구현하면 어떤 제약이 생기나

실습 목표

항목	내용
실행 명령	npm run exercise:s15:3
핵심 과제	implements IBlockchainAdapter 선언 후 TypeScript 컴파일 통과
구현 방식	모든 메서드를 Stub(고정값 반환)으로 채움
chainType	'XRPL' as const — 체인 식별자 필드 필수

메서드 하나라도 빠지면 **컴파일 에러**
→ 런타임 전에 누락 감지 = TypeScript의 핵심 가치

결과 분석 — Stub 반환값의 의미

메서드	Stub 반환값	이유
isConnected	true	항상 연결됨 가정
getBlockNumber	99_999_999	XRPL은 ledger index
mintNFT	고정 txHash	페이크 영수증
getBalance	0n	잔액 조회 미구현
getReceipt	null	PENDING 없음 가정
queryEvents	[]	이벤트 없음 가정

Stub = 컴파일 통과용 최소 구현. Phase 3에서 실제 XRPL SDK 로직으로 교체.

[3] 교체 시뮬레이션 — 결과 분석

같은 호출 코드 → EVM / XRPL 두 어댑터 동시 실행

실행 결과

어댑터	isConnected	blockNumber
[EVM]	true	84XXXXX (Sepolia 실시간)
[XRPL]	true	99,999,999 (Stub 고정값)

호출 코드는 완전히 동일

```
adapter.isConnected()
```

```
adapter.getBlockNumber()
```

for 루프 안에서 어댑터를 교체해도 코드 한 줄 변경 없음

이 실습이 증명하는 것

질문	답
체인을 바꾸려면 서비스 코드도 바뀌어야 하나?	No — 어댑터 교체만
XRPL은 블록이 없는데?	ledger index를 blockNumber로 매핑
chainType은 왜 필요한가?	Registry 라우팅 시 체인 식 별자
Stub 값이 99_999_999인 이유?	EVM 블록(8M대)과 명확히 구분

S14 Strategy Pattern 검증 완료 — IBlockchainAdapter 인터페이스가 체인 종류와 비즈니스 로직을 완전히 분리함을 숫자로 확인

[4] read-only 모드 · [5] mintNFT + getBalance

privateKey 유무가 동작에 어떤 차이를 만드는가

[4] read-only — 에러 결과 분석

항목	내용
실행 명령	npm run exercise:s15:4
시도	privateKey 없이 mintNFT 호출
에러	Wallet not initialized
발생 위치	생성자에서 this.wallet = null → mintNFT 진입 즉시 throw

설계 의도 — TX를 보내려면 반드시 서명 키가 있어야 한다.
실수로 키 없이 생성해도 읽기는 되고 쓰기만 차단 → 최소 권한 원칙

[5] mintNFT + getBalance — 결과 분석

항목	내용
실행 명령	npm run exercise:s15:5
전제	MOCK_CONTRACT_ADDR + EVM_SIGNER_KEY 환경변수 필요
receipt 필드	txHash · blockNumber · blockHash · status · timestamp
balance 타입	bigint — JS Number 범위 초과 방지

balance가 즉시 1인 이유

mintNFT tx가 블록에 포함된 직후 getBalance → 온체인 상태 반영

(tx.wait() 내부적으로 포함 확인 후 반환)

[7] queryEvents — 이벤트 조회 결과 분석

TransferSingle 이벤트 · S15 실습 정리

[7] queryEvents — 결과 분석

항목	내용
실행 명령	<code>npm run exercise:s15:7</code>
조회 범위	<code>currentBlock - 1000 ~ currentBlock</code> (최근 1000블록)
필터 이벤트	TransferSingle — ERC1155 민팅·전송 시 항상 발생
ABI 방식	Human-readable ABI — 이벤트 시그니처 문자열만 넘기면 내부에서 파싱

ChainEvent 객체 구조

event: 이벤트명 · args: {operator, from, to, id, value}

blockNumber · txHash · logIndex

[5]에서 mintNFT 실행 후 바로 [7] 실행하면 방금 민팅한 이벤트가 배열에 포함됨 → 온체인 상태 추적 흐름 확인

S15 실습 체크리스트

섹션	핵심 확인 포인트
[1]	isConnected: true — Provider 연결 확인
[2]	실시간 블록 번호 — 매 실행마다 다름
[3]	EVM/XRPL 두 어댑터 동일 인터페이스 호출
[4]	Wallet not initialized 에러 — 설계 검증
[5]	receipt.status: 'success' + balance: 1n
[7]	TransferSingle ChainEvent 배열 출력

S15 실습 완료

EVMAdapter 구현 분석 → XRPLMockAdapter 교체 → Strategy Pattern 검증 → 온체인 이벤트 조회까지 전 사이클 완료